Module 11 — Cost, Latency & GPU Optimization

> **Goal:** Cut cost per useful token by 3–10× without breaking quality or SLOs. Covers semantic caching (Redis + vector match), dynamic model cascading/routing, spot-instance management, tensor vs pipeline parallelism, serverless GPU endpoints (Modal, RunPod, AWS Bedrock), and the unit economics to justify each choice.

Table of Contents

1. Module Overview & First-Principles Architecture
2. Mathematical Foundations
3. Production-Ready Code
4. Hard-Learned Production Failures
5. Interview Questions

---

1. Module Overview & First-Principles Architecture

1.1 The cost stack — where the money goes

```
Total cost = Compute (GPU-hours) + Tokens (API fees) + Data (vector DB, storage, egress)
           + Eval/judge calls + Observability + Engineering time + Idle capacity
```

For self-hosted LLM serving the dominant lever is **utilisation**: a GPU costs the same per hour whether it serves 5 or 5,000 tokens/s. For API usage the dominant levers are **tokens** (reduce, cache, route to cheaper models).

Optimisation ladder (apply top-down; each step is cheaper and less risky than the next):

1. **Don't call the LLM** (deterministic code, exact cache, semantic cache, FAQ short-circuit).
2. **Send fewer tokens** (prompt compression, better retrieval/reranking, summarised memory, shorter outputs, prefix caching).
3. **Use a cheaper model when sufficient** (routing/cascades, distillation, fine-tuned small models).
4. **Make each GPU-second produce more tokens** (quantisation, batching, speculative decoding, FP8, KV-cache efficiency).
5. **Pay less per GPU-hour** (spot/preemptible, reserved/savings plans, right-sizing, serverless scale-to-zero).
6. **Schedule smartly** (batch/offline work off-peak on spot, priority tiers).

1.2 Reference cost-optimised request path

```
flowchart LR
  R[Request] --> E[Exact cache<br/>hash(prompt+params)]
  E -- hit --> OUT[Response]
  E -- miss --> S[Semantic cache<br/>Redis vector search]
  S -- "sim >= τ & validated" --> OUT
  S -- miss --> RT[Router<br/>classifier / cascade]
  RT -- easy --> M8[8B model]
  RT -- medium --> M70[70B model]
```

```
RT -- hard --> FR[Frontier / 100B+]
M8 --> V{Confidence /<br/>verifier OK?}
V -- yes --> W[Write-back to cache] --> OUT
V -- no  --> M70
M70 --> W
FR --> W
```

1.3 Semantic caching

An exact cache keys on a hash of the full prompt; hit rate on natural-language traffic is typically < 5 %. A **semantic cache** embeds the query and returns a stored answer when cosine similarity ≥ threshold τ — hit rates of 20–60 % for FAQ-style/support/documentation workloads.

Design decisions:

Decision	Options	Guidance
Key	Query only vs query + context fingerprint	If answers depend on retrieved docs/user/tenant/time, **namespace the key** (tenant, index version, prompt version, model) — otherwise you serve stale or cross-tenant answers
Similarity	Cosine on small embedder (bge-small, 384-d)	Threshold 0.90–0.97; calibrate on labelled paraphrase/non-paraphrase pairs
Validation	Reranker/cross-encoder or LLM check on borderline hits	Reduces false hits at $\tau \in$ [0.85, 0.95]
TTL / invalidation	Time-based + event-based (doc updated → purge by `doc_id` tags)	Short TTL for volatile data
Safety	Never cache outputs flagged by guardrails, personalised, or containing PII	Cache only after output rails pass
Streaming	Replay cached text with simulated token timing	Preserves UX
Storage	Redis Stack (RediSearch HNSW), Qdrant, or GPTCache	Redis gives ms latency + TTL

Risks: **false positives** (semantically close but different intent: "How do I cancel *my* order?" vs "…*an* order?", "limit of 5" vs "limit of 50" — embeddings blur numbers/negation), **cache poisoning** (attacker seeds wrong answers), **privacy leakage** across tenants.

1.4 Dynamic model cascading / routing

Strategies:

Strategy	Mechanism	Pros / Cons
Static rules	Route by feature/endpoint/length	Simple, predictable; coarse
Classifier router	Small model predicts difficulty or "needs big model" from the prompt (RouteLLM-style trained on preference data; or embedding + logistic regression)	Single call latency; needs labelled data; quality ↔ cost tunable by threshold
Cascade (try cheap first, escalate)	Small model answers; confidence/verifier decides to escalate	Cost-efficient on easy-heavy traffic; adds latency on escalations (~+1×small)
Speculative cascade	Run small and big concurrently, cancel big if small is confident	Latency-safe, wastes some compute

Strategy	Mechanism	Pros / Cons
Mixture routing by capability	Code → code-tuned model, math → reasoning model, multilingual → X	Needs intent classification
Budget-aware	Per-tenant spend caps degrade to smaller model	Protects margin

Confidence signals for escalation: mean token log-prob / min-prob, self-consistency disagreement (2–3 samples), verifier model score, rule checks (JSON valid, citations present, retrieved-context score), refusal/"I'm not sure" detection, answer length anomalies.

1.5 Parallelism: tensor vs pipeline (serving view)

	Tensor parallel (TP)	Pipeline parallel (PP)
Splits	Each layer's matrices across GPUs	Contiguous layer groups per GPU/node
Comm	2 all-reduces/layer, per token → needs NVLink/NVSwitch	Point-to-point activations once per stage boundary
Latency	**Reduces** per-token latency (parallel compute, more aggregate memory BW)	**Adds** latency (sequential stages) unless many micro-batches fill the pipe
Throughput	Good; limited by comm at high TP	Good for large batch; bubbles at low batch
Best for	Within a node (NVLink), low-latency	Across nodes / slow interconnect, huge models, throughput
Typical	TP=2–8 within DGX	TP=8 × PP=2–4 across nodes (e.g., 405B BF16)

Rule: **TP inside the NVLink domain, PP across nodes, replicas (DP) to scale out.** Prefer the *smallest* TP that fits weights + KV; quantise before sharding more.

1.6 Spot / preemptible instances

Cloud spot GPUs are 50–90 % cheaper but can be reclaimed with 30 s–2 min notice (AWS 2 min, GCP 30 s, Azure 30 s). Inference on spot requires: - **Stateless replicas** behind a load balancer; drain on interruption notice. - **Diversified pools** (multiple instance types/AZs/regions) and **capacity fallback** to on-demand. - **Fast cold start** (weights on local NVMe/image, streamed loading) — target < 2 min. - **Graceful degradation** path (route to a smaller model or serverless endpoint when capacity drops). - **Checkpointing** for training/fine-tuning (save every N minutes; resume). - Mix: reserved/committed baseline (steady load) + spot for burst/batch.

1.7 Serverless GPU endpoints

Option	Model	Strengths	Watch-outs
Modal	Python-native serverless containers, per-second billing, scale-to-zero, GPU snapshots for fast cold start	Great DX, bursty workloads, batch jobs, fine-tune runs	Cold starts (seconds→tens of seconds); vendor lock-in; limits on long-lived streaming
RunPod (Serverless / Pods)	Serverless workers with vLLM templates; cheaper community GPUs	Low $/GPU-hr, flexible GPUs	Variable availability/reliability; enterprise controls vary
AWS Bedrock	Managed API to hosted FMs (Claude, Llama, Mistral, Nova…), provisioned throughput, custom model import, batch inference	No GPU ops, compliance, IAM/VPC, guardrails	Per-token cost at scale > self-host; model availability/quota
SageMaker / Vertex / Azure ML endpoints	Managed hosting of your container	Autoscaling, VPC	Min instance cost unless serverless/async

Option	Model	Strengths	Watch-outs
Together / Fireworks / Baseten	Hosted open models	Fast, cheap per token	Data-processing terms

Break-even: serverless wins at **low or spiky utilisation**; dedicated wins at **sustained high utilisation** (see §2.3).

---

2. Mathematical Foundations & Formulas

2.1 Cost per million tokens (self-hosted)

```
$/1M_out = (GPU_$/hr × n_gpu) / (TPS_sys × 3600 × U) × 10■
```

U = fraction of the hour the fleet is actually busy (billing utilisation). Doubling U halves unit cost.

Example (illustrative, perishable inputs: the formula is the takeaway): 1×H100 at $3.00/hr, sustained 4,500 tok/s when busy: - U = 100%: $0.185 / 1M - U = 30%: $0.617 / 1M - Compare API price $0.60 / 1M: self-hosting only wins when U ■ 31 %.

2.2 Semantic cache economics

Hit rate h, cache lookup cost c_c (embedding + vector search ≈ 8–15 ms, ~$0), LLM cost c_L, false-hit rate f, cost of a wrong cached answer L_w:

```
E[cost/request] = c_c + (1 – h)·c_L + h·f·L_w
Savings = h·c_L – c_c – h·f·L_w
```

Latency: E[T] = T_c + (1–h)·T_L. With h = 0.35, T_L = 2.5 s, T_c = 15 ms: E[T] = 1.64 s (–34 %). Pick τ to keep f ≤ 0.5 % on a labelled set even if it lowers h.

2.3 Dedicated vs serverless break-even

Dedicated: fixed $D/hr. Serverless: $S per busy second (premium k = S·3600/D, rule of thumb 1.5–4×, perishable: check current pricing), billed only while serving. Break-even utilisation:

```
U* = 1/k
```

If serverless costs k=3× per busy-second, dedicated is cheaper when the GPU would be busy > 33 % of the time. Add cold-start time t_cs penalty: wasted t_cs × rate per scale-up event — frequent scale-to-zero oscillations destroy savings.

2.4 Cascade cost model

Fraction e of requests escalate from the small model (cost c_s, latency T_s) to the large (c_l, T_l); router/verifier cost c_v:

```
E[cost] = c_s + c_v + e·c_l
E[T]    = T_s + T_v + e·T_l
Break-even vs always-large:  c_s + c_v + e·c_l < c_l  ⇔  e < 1 – (c_s + c_v)/c_l
```

8B vs 70B: c_s/c_l ≈ 0.1 → cascade pays if e < ~0.88 (allowing for c_v). But quality: overall accuracy A = (1–e)·a_s|accepted + e·a_l; require A ≥ A_large – ε. Choose the threshold on the **cost–quality Pareto curve** (published routing work, RouteLLM among it, reports large cost savings at a small quality loss on public benchmarks; the figure depends on the model pair and workload, so measure it on your own traffic).

2.5 Routing as a threshold problem

Router outputs p = P(small model suffices). Route to small iff p ≥ θ. Raising θ improves quality, lowers savings. Evaluate with: **quality retention** Q = A_routed / A_large, **call fraction** to large e(θ), and plot Q vs e. A random router yields a straight line; a good router bows above it. Report APGR (average performance gap recovered).

2.6 Spot economics with interruptions

Spot price p_s, on-demand p_o, interruption rate λ per hour, recovery overhead t_r (reload + redo), fraction of time on fallback on-demand ϕ:

```
Effective $/hr = (1 – ϕ)·p_s·(1 + λ·t_r) + ϕ·p_o
```

With $p_s = 0.35$ p_o, $\lambda = 0.05$/hr, $t_r = 0.05$ hr (3 min), $\phi = 0.15$: $0.85 \times 0.35 \times 1.0025 + 0.15 = 0.448$ p_o → ~55 % savings. For training: add lost work $\lambda \times$ (checkpoint_interval/2).

2.7 Tensor-parallel scaling efficiency

Per-token latency with TP degree t:

```
ITL(t) ≈ (W/t + KV/t)/BW_hbm  +  2·L·(α + (t–1)/t · V_act / BW_link)
```

α = all-reduce latency (≈ 5–15 µs NVLink; 20–50 µs PCIe/IB), $V_{act} = b \cdot d \cdot$ bytes. Speedup saturates when comm ≈ compute: for 8B at TP=8 on NVLink, the memory term (≈ 2 GB/GPU → 0.7 ms) is comparable to $2 \cdot 32 \cdot \alpha \approx 0.5$–1 ms, so **TP>2 gives little benefit for small models** while 70B+ benefits strongly.

2.8 Pipeline bubble

With p stages and m micro-batches, bubble fraction = $(p - 1)/(m + p - 1)$. p=4, m=4 → 43 % idle; m=32 → 8.6 %. For inference, continuous batching naturally supplies many in-flight micro-batches at high load; at low load PP is inefficient and increases TTFT by (p–1)×stage_latency.

2.9 Prompt-caching & compression savings

Provider prompt caching: cached tokens billed at r_c (0.1–0.5) of input price; for a prompt with cached fraction f:

```
input_cost = in_tokens·p_in·( f·r_c + (1 – f) )
```

f=0.8, r_c=0.1 → 0.28 of baseline (72 % off). Prompt compression (LLMLingua-style) with ratio ρ reduces input tokens by $1–\rho$ at some quality risk — evaluate per task.

2.10 Batch (offline) vs interactive pricing

Batch APIs offer ~50 % discount for 24 h SLA. If a fraction β of workload is deferrable: savings = $\beta \times 0.5$ on that part. Self-hosted: schedule batch to fill valleys of diurnal traffic — raising U — and run on spot.

---

3. Production-Ready Executable Code

3.1 Requirements

```
# requirements-cost.txt
redis>=5.0
numpy>=1.26
httpx>=0.27
openai>=1.40
fastapi>=0.112
uvicorn>=0.30
pydantic>=2.7
scikit-learn>=1.5
sentence-transformers>=3.0
prometheus-client>=0.20
modal>=0.64
boto3>=1.35
tenacity>=8.3
```

3.2 Semantic Cache on Redis Stack (RediSearch HNSW) with Namespacing, TTL, Validation, Safe Write-Back

```
"""Production semantic cache: Redis Stack vector index,
 tenant/version namespacing, TTL, tag-based invalidation, borderline validation."""
from __future__ import annotations

import hashlib
import json
import time
from dataclasses import dataclass
from typing import Awaitable, Callable

import numpy as np
import redis.asyncio as redis
from redis.commands.search.field import NumericField, TagField, TextField, VectorField
from redis.commands.search.indexDefinition import IndexDefinition, IndexType
from redis.commands.search.query import Query
from sentence_transformers import SentenceTransformer

DIM = 384
INDEX = "idx:semcache"
_embedder = SentenceTransformer("BAAI/bge-small-en-v1.5", device="cpu")


def embed(text: str) -> np.ndarray:
    return _embedder.encode(text, normalize_embeddings=True).astype(np.float32)


@dataclass(frozen=True)
class CacheCfg:
    tau_hit: float = 0.94          # accept without validation
    tau_validate: float = 0.88     # borderline: ask validator
    ttl_s: int = 6 * 3600
    max_answer_chars: int = 8000


class SemanticCache:
    def __init__(self, r: redis.Redis, cfg: CacheCfg = CacheCfg()) -> None:
        self.r, self.cfg = r, cfg

    async def setup(self) -> None:
        try:
            await self.r.ft(INDEX).info()
        except Exception:
            schema = (
                TagField("ns"),                  # tenant|model|prompt_v|index_v namespace  =&
gt; hard isolation
                TagField("tags"),                # doc ids etc. for invalidation
                TextField("query"),
                NumericField("created"),
                VectorField("vec", "HNSW", {"TYPE": "FLOAT32", &q
uot;DIM": DIM, "DISTANCE_METRIC": "COSINE", "M": 16, "EF_CONSTRUC
TION": 200}),
            )
            await self.r.ft(INDEX).create_index(schema, definition=IndexDefinition(prefix=["sc:"
;], index_type=IndexType.HASH))

    @staticmethod
    def namespace(tenant: str, model: str, prompt_v: int, index_v: str) -> str:
        return hashlib.sha1(f"{tenant}|{model}|{prompt_v}|{index_v}".encode()).hexdigest()[:16]

    async def lookup(self, ns: str, query: str, validator: Callable[[str, str], Awaitable[bool]] | None =
 None) -> tuple[str | None, float]:
        v = embed(query)
```

```
        q = (Query(f"(@ns:{{{ns}}})=>[KNN 3 @vec $v AS dist]").sort_by("dist").ret
urn_fields("dist", "query", "answer").dialect(2))
        res = await self.r.ft(INDEX).search(q, query_params={"v": v.tobytes()})
        for doc in res.docs:
            sim = 1.0 - float(doc.dist)                    # cosine distance -> similarity
            if sim >= self.cfg.tau_hit:
                return doc.answer, sim
            if sim >= self.cfg.tau_validate and validator is not None and await validator(query, doc.q
uery):
                return doc.answer, sim
        return None, 0.0

    async def put(self, ns: str, query: str, answer: str, tags: list[str]) -> None:
        if len(answer) > self.cfg.max_answer_chars:
            return
        key = f"sc:{ns}:{hashlib.sha1(query.encode()).hexdigest()}"
        await self.r.hset(key, mapping={"ns": ns, "query": query, "answer":
 answer, "tags": ",".join(tags) or "none",
                                        "created": int(time.time()), "vec": embed(que
ry).tobytes()})
        await self.r.expire(key, self.cfg.ttl_s)

    async def invalidate_tag(self, tag: str) -> int:
        """Purge all entries derived from a changed document."""
        q = Query(f"@tags:{{{tag}}}").return_fields("created").paging(0, 1000)
        res = await self.r.ft(INDEX).search(q)
        if res.docs:
            await self.r.delete(*[d.id for d in res.docs])
        return len(res.docs)
```

Validation function for borderline hits (cheap model or cross-encoder deciding "same intent?"):

```
async def same_intent(a: str, b: str) -> bool:
    from openai import AsyncOpenAI
    c = AsyncOpenAI(base_url="http://localhost:8003/v1", api_key="EMPTY")      # smal
l 1-3B model
    r = await c.chat.completions.create(model="llm-tiny", temperature=0, max_tokens=3, messages
=[{
        "role": "user", "content": "Do these two questions have EXACTL
Y the same intent and the same specifics "
        f"(numbers, names, negations)? Answer Yes or No.\nA: {a}\nB: {b}"}])
    return (r.choices[0].message.content or "").strip().lower().startswith("yes")
```

Calibrating τ: label $\geq$ 500 pairs (paraphrase vs. non-paraphrase, including hard negatives with swapped numbers/negation); sweep τ and choose the smallest value with false-hit rate $\leq$ 0.5 %.

```
def pick_tau(sims: np.ndarray, is_same: np.ndarray, max_fpr: float = 0
.005) -> float:
    for tau in np.arange(0.80, 0.995, 0.005):
        pred = sims >= tau
        fpr = (pred & ~is_same).sum() / max(1, (~is_same).sum())
        if fpr <= max_fpr:
            return float(tau)
    return 0.99
```

3.3 Cached-LLM Gateway with Exact Cache, Semantic Cache, Streaming Replay, Metrics

```
from __future__ import annotations

import asyncio
import hashlib
import json
```

```
from typing import AsyncIterator

import redis.asyncio as redis
from fastapi import FastAPI
from fastapi.responses import StreamingResponse
from pydantic import BaseModel
from prometheus_client import Counter, make_asgi_app

from semantic_cache import SemanticCache, same_intent       # §3.2
from router import route_and_generate                       # §3.4

HITS = Counter("cache_requests_total", "cache outcomes", ["layer", "ou
tcome"])
app = FastAPI(); app.mount("/metrics", make_asgi_app())
r = redis.Redis(); sc = SemanticCache(r)


class Req(BaseModel):
    tenant: str
    query: str
    context_fingerprint: str = ""        # hash of retrieved doc ids+versions, if answer depend
s on RAG
    prompt_v: int = 23
    index_v: str = "kb_v3"
    cacheable: bool = True               # False for personalised / PII / tool-using requests


@app.on_event("startup")
async def _s() -> None:
    await sc.setup()


async def replay(text: str, chunk: int = 4, delay: float = 0.012) -> AsyncIterator[bytes]:
    words = text.split(" ")
    for i in range(0, len(words), chunk):
        yield ("data: " + json.dumps({"delta": " ".join(words[i:i+chunk]) +
 " "}) + "\n\n").encode()
        await asyncio.sleep(delay)
    yield b"data: [DONE]\n\n"


@app.post("/ask")
async def ask(req: Req) -> StreamingResponse:
    ns = sc.namespace(req.tenant, "family-main", req.prompt_v, req.index_v + req.context_finger
print)
    if req.cacheable:
        exact_key = "ec:" + hashlib.sha256(f"{ns}|{req.query}".encode()).hexdigest()
        if (v := await r.get(exact_key)) is not None:
            HITS.labels("exact", "hit").inc(); return StreamingResponse(replay(v.deco
de()), media_type="text/event-stream")
        ans, sim = await sc.lookup(ns, req.query, validator=same_intent)
        if ans:
            HITS.labels("semantic", "hit").inc(); return StreamingResponse(replay(ans
), media_type="text/event-stream")
        HITS.labels("semantic", "miss").inc()
    text, meta = await route_and_generate(req.query)
    if req.cacheable and meta["safe_to_cache"]:
        await r.set("ec:" + hashlib.sha256(f"{ns}|{req.query}".encode()).hexdigest(),
 text, ex=3600)
        await sc.put(ns, req.query, text, tags=meta.get("doc_ids", []))
    return StreamingResponse(replay(text), media_type="text/event-stream")
```

3.4 Router + Cascade (embedding classifier, confidence-based escalation, budget awareness)

```
"""Dynamic model routing: (1) learned difficulty router
, (2) cascade with confidence + verifier, (3) tenant budget degrade."""
from __future__ import annotations

import json
import math
import pickle
from dataclasses import dataclass
from pathlib import Path
from typing import Any

import numpy as np
from openai import AsyncOpenAI

small = AsyncOpenAI(base_url="http://vllm-8b:8000/v1", api_key="EMPTY")
mid = AsyncOpenAI(base_url="http://vllm-70b:8000/v1", api_key="EMPTY")
big = AsyncOpenAI(base_url="https://frontier.example/v1", api_key="...")

COST = {"small": 0.05, "mid": 0.45, "big": 3.00}                    # $/
1M output tokens (illustrative)
THETA_EASY, THETA_MID = 0.80, 0.45
_router = pickle.loads(Path("router_lr.pkl").read_bytes())            # sklearn LogisticRegres
sion trained on embeddings


def difficulty_probs(q_vec: np.ndarray) -> dict[str, float]:
    p = _router.predict_proba(q_vec.reshape(1, -1))[0]                # classes: [easy, medium, hard]
    return {"easy": float(p[0]), "medium": float(p[1]), "hard": float(p[2])
}


@dataclass
class Gen:
    text: str
    mean_logprob: float
    min_logprob: float
    finish: str


async def call(client: AsyncOpenAI, model: str, q: str, max_tokens: int = 512) -> Gen:
    r = await client.chat.completions.create(model=model, temperature=0, max_tokens=max_tokens, logprobs=
True, top_logprobs=1,
                                             messages=[{"role": "system", "conten
t": "Be accurate and concise."}, {"role": "user", "content":
 q}])
    ch = r.choices[0]
    lps = [t.logprob for t in (ch.logprobs.content or [])] or [0.0]
    return Gen(ch.message.content or "", float(np.mean(lps)), float(np.min(lps)), ch.finish_rea
son)


def confident(g: Gen, q: str) -> bool:
    """Cheap, calibrated-on-eval confidence gate. Tune thresholds on a labelled set to hit
 target accuracy of accepted answers."""
    if g.finish == "length" or len(g.text.strip()) < 2:
        return False
    if any(m in g.text.lower() for m in ("i'm not sure", "i cannot", "as an ai&q
uot;, "unclear")):
        return False
    return g.mean_logprob > -0.35 and g.min_logprob > -4.0


async def verifier_ok(q: str, a: str) -> bool:
    r = await small.chat.completions.create(model="llm-8b", temperature=0, max_tokens=2, messag
es=[{"role": "user", "content":
        f"Question: {q}\nAnswer: {a}\nIs the answer correct, complete and directly responsive? Reply
```

```
 Yes or No."}])
    return (r.choices[0].message.content or "").strip().lower().startswith("y")


async def route_and_generate(q: str, q_vec: np.ndarray | None = None, budget_exhausted: bool = False) -&g
t; tuple[str, dict[str, Any]]:
    meta: dict[str, Any] = {"path": [], "safe_to_cache": True}
    if budget_exhausted:                                    # tenant over budget -> degrade
, don't fail
        g = await call(small, "llm-8b", q); meta["path"] = ["small(degraded)&quo
t;]; return g.text, meta
    p = difficulty_probs(q_vec) if q_vec is not None else {"easy": 0.5, "medium": 0.4
, "hard": 0.1}
    if p["hard"] > 0.5:
        g = await call(big, "frontier", q); meta["path"] = ["big(routed)"];
 return g.text, meta
    if p["easy"] >= THETA_EASY:
        g = await call(small, "llm-8b", q); meta["path"].append("small")
        if confident(g, q) and await verifier_ok(q, g.text):
            return g.text, meta
    g = await call(mid, "llm-70b", q); meta["path"].append("mid")
    if confident(g, q) or p["medium"] >= THETA_MID:
        return g.text, meta
    g = await call(big, "frontier", q); meta["path"].append("big(escalated)"
;)
    return g.text, meta
```

Train the router (preference/outcome data: for each prompt, did the small model's answer match the large model's per judge?):

```
from __future__ import annotations

import pickle

import numpy as np
from sentence_transformers import SentenceTransformer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report
from sklearn.model_selection import train_test_split

emb = SentenceTransformer("BAAI/bge-small-en-v1.5")


def train(prompts: list[str], labels: list[int]) -> None:
    """labels: 0=small suffices, 1=needs mid, 2=needs big  (derived from judge comparisons
 vs reference answers)."""
    X = emb.encode(prompts, normalize_embeddings=True, batch_size=128)
    Xtr, Xte, ytr, yte = train_test_split(X, labels, test_size=0.2, stratify=labels, random_state=0)
    clf = LogisticRegression(max_iter=2000, class_weight="balanced", C=2.0).fit(Xtr, ytr)
    print(classification_report(yte, clf.predict(Xte)))
    pickle.dump(clf, open("router_lr.pkl", "wb"))


def pareto(clf, X, quality_small: np.ndarray, quality_large: np.ndarray, cost_ratio: float = 0.1) -> l
ist[tuple[float, float, float]]:
    """Sweep threshold: returns (theta, fraction_to_large, quality_retention).""
"
    p_small_ok = clf.predict_proba(X)[:, 0]
    out = []
    for th in np.linspace(0.05, 0.95, 19):
        to_small = p_small_ok >= th
        q = np.where(to_small, quality_small, quality_large).mean() / quality_large.mean()
        out.append((float(th), float(1 - to_small.mean()), float(q)))
    return out
```

3.5 Prompt-Cache-friendly Prompt Compression & Output-Length Control

```
"""Token diet: cap output length by intent, drop low-va
lue retrieved text, and ask for concise formats."""
from __future__ import annotations

MAX_OUT = {"faq": 200, "summary": 400, "analysis": 900, "code": 1
200}

def output_budget(intent: str, user_requested: int | None) -> int:
    base = MAX_OUT.get(intent, 500)
    return min(base, user_requested) if user_requested else base


def trim_chunks(chunks: list[tuple[str, float]], max_tokens: int, count) -> list[str]:
    """chunks sorted by rerank score desc; keep until budget, stop when marginal score <
; 40% of the top score."""
    if not chunks:
        return []
    top = chunks[0][1]; kept, used = [], 0
    for text, score in chunks:
        t = count(text)
        if used + t > max_tokens or score < 0.4 * top:
            break
        kept.append(text); used += t
    return kept
```

3.6 Kubernetes: Spot GPU Node Pool, Karpenter NodePool, Interruption-aware Draining

```
# Karpenter NodePool: prefer spot, fall back to on-demand, diversify ins
tance families
apiVersion: karpenter.sh/v1
kind: NodePool
metadata: { name: gpu-inference }
spec:
  template:
    spec:
      nodeClassRef: { group: karpenter.k8s.aws, kind: EC2NodeClass, name: gpu }
      requirements:
        - { key: karpenter.sh/capacity-type, operator: In, values: ["spot", "on-demand&quo
t;] }
        - { key: karpenter.k8s.aws/instance-family, operator: In, values: ["p5", "p4d"
;, "g6e", "g5"] }
        - { key: karpenter.k8s.aws/instance-gpu-count, operator: In, values: ["1", "2"
;, "4", "8"] }
        - { key: topology.kubernetes.io/zone, operator: In, values: ["us-east-1a", "us-eas
t-1b", "us-east-1c"] }
      taints: [{ key: nvidia.com/gpu, value: "true", effect: NoSchedule }]
      expireAfter: 168h
  disruption:
    consolidationPolicy: WhenEmptyOrUnderutilized
    consolidateAfter: 10m
  limits: { nvidia.com/gpu: 64 }
---
# Interruption notice handler: AWS Node Termination Handler (queue mode) cordons+drains within the 2-minu
te warning
apiVersion: v1
kind: ConfigMap
metadata: { name: nth-config }
data:
  ENABLE_SQS_TERMINATION_DRAINING: "true"
  ENABLE_SPOT_INTERRUPTION_DRAINING: "true"
  ENABLE_REBALANCE_DRAINING: "true"
---
# Pods: tolerate spot, spread across zones/nodes, short grace so drain finishes inside the 2-minute notic
```

```
e
apiVersion: apps/v1
kind: Deployment
metadata: { name: vllm-spot }
spec:
  replicas: 6
  selector: { matchLabels: { app: vllm-spot } }
  template:
    metadata: { labels: { app: vllm-spot } }
    spec:
      terminationGracePeriodSeconds: 100
      topologySpreadConstraints:
        - { maxSkew: 1, topologyKey: topology.kubernetes.io/zone, whenUnsatisfiable: ScheduleAnyway, labe
lSelector: { matchLabels: { app: vllm-spot } } }
      tolerations: [{ key: nvidia.com/gpu, operator: Exists }]
      containers:
        - name: vllm
          image: vllm/vllm-openai:v0.10.1
          args: ["--model=/models/llama31-8b-fp8", "--max-model-len=8192", "--en
able-prefix-caching", "--enable-chunked-prefill"]
          resources: { limits: { nvidia.com/gpu: 1 } }
          volumeMounts: [{ name: nvme, mountPath: /models }]   # weights on node-local NVMe (pre-baked AM
I / init-container sync)
      volumes: [{ name: nvme, hostPath: { path: /mnt/nvme/models } }]
---
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata: { name: vllm-spot-pdb }
spec: { maxUnavailable: 25%, selector: { matchLabels: { app: vllm-spot } } }
```

Two-tier capacity: a small on-demand baseline deployment (vllm-ondemand, 40 % of p50 demand) and the spot deployment behind the same Service; gateway weights traffic and **sheds to the cheaper small model** when healthy replicas fall below a floor.

3.7 Spot-Interruption Watcher (cloud-agnostic sidecar pattern, AWS IMDSv2 shown)

```
"""Poll IMDS for a spot interruption notice; flip readi
ness to NotReady immediately and drain streams."""
from __future__ import annotations

import os
import signal
import time

import httpx

IMDS = "http://169.254.169.254"


def token() -> str:
    return httpx.put(f"{IMDS}/latest/api/token", headers={"X-aws-ec2-metadata-token-ttl-se
conds": "60"}, timeout=2).text


def interrupted() -> bool:
    r = httpx.get(f"{IMDS}/latest/meta-data/spot/instance-action", headers={"X-aws-ec2-met
adata-token": token()}, timeout=2)
    return r.status_code == 200


if __name__ == "__main__":
    while True:
        try:
            if interrupted():
                open("/tmp/draining", "w").write("1")        # readiness pr
obe checks this file => LB stops sending new requests
```

```
            time.sleep(90)                          # let in-flight streams finish
            os.kill(1, signal.SIGTERM)
            break
        except Exception:
            pass
        time.sleep(5)
```

3.8 Modal: Serverless vLLM Endpoint with GPU Snapshotting and Scale-to-Zero

```
"""modal deploy modal_vllm.py  — OpenAI-compatible endp
oint that scales to zero, per-second billing."""
from __future__ import annotations

import subprocess

import modal

MODEL = "meta-llama/Llama-3.1-8B-Instruct"
image = (modal.Image.debian_slim(python_version="3.11")
         .pip_install("vllm==0.10.1", "huggingface_hub[hf_transfer]")
         .env({"HF_HUB_ENABLE_HF_TRANSFER": "1", "VLLM_USE_V1": "1&quo
t;}))
vol = modal.Volume.from_name("hf-cache", create_if_missing=True)
app = modal.App("llm-serverless", image=image)


@app.function(gpu="H100", volumes={"/root/.cache/huggingface": vol}, secrets=[modal.S
ecret.from_name("hf-token")],
              scaledown_window=300,                 # keep warm 5 min after last request (avoid thrash vs
 cold start cost)
              min_containers=0, max_containers=8,
              timeout=60 * 30)
@modal.concurrent(max_inputs=64)                    # many concurrent requests per container: vLLM batche
s them
@modal.web_server(port=8000, startup_timeout=600)
def serve() -> None:
    subprocess.Popen(["vllm", "serve", MODEL, "--served-model-name", "
base", "--max-model-len", "8192",
                      "--gpu-memory-utilization", "0.90", "--enable-prefix-cachi
ng", "--enable-chunked-prefill",
                      "--host", "0.0.0.0", "--port", "8000"])


@app.function(gpu="A10G", schedule=modal.Cron("0 2 * * *"), timeout=3600)
def nightly_batch_job() -> None:
    """Offline eval/synthetic data generation on cheaper GPUs, billed only while running.&
quot;""
    from vllm import LLM, SamplingParams

    llm = LLM(model="Qwen/Qwen2.5-7B-Instruct-AWQ", quantization="awq_marlin", max_mo
del_len=4096,
              gpu_memory_utilization=0.9, enable_prefix_caching=True)
    prompts = [f"Write one realistic customer-support question about topic #{i}." for i in rang
e(2000)]
    outs = llm.generate(prompts, SamplingParams(temperature=0.9, max_tokens=64))
    with open("/root/.cache/huggingface/synthetic_questions.jsonl", "w") as f:
        for o in outs:
            f.write(o.outputs[0].text.strip().replace("\n", " ") + "\n")
```

Cold-start tuning: weights in a Volume (or baked into the image), `--enforce-eager` off with CUDA-graph snapshot enabled where supported, `scaledown_window` sized to the typical inter-request gap; set `min_containers=1` during business hours via a scheduled function to avoid user-visible cold starts.

3.9 RunPod Serverless Worker (vLLM) Handler

```
"""RunPod serverless handler wrapping vLLM's AsyncLLMEn
gine; deployed as a container image."""
from __future__ import annotations

import runpod
from vllm import AsyncEngineArgs, AsyncLLMEngine, SamplingParams

engine = AsyncLLMEngine.from_engine_args(AsyncEngineArgs(model="/runpod-volume/llama31-8b-awq",
 quantization="awq_marlin",
                                         max_model_len=8192, gpu_memory_utilization=0.9,
 enable_prefix_caching=True))


async def handler(job: dict):
    inp = job["input"]
    sp = SamplingParams(temperature=inp.get("temperature", 0.2), max_tokens=min(inp.get("m
ax_tokens", 256), 1024))
    last = ""
    async for out in engine.generate(inp["prompt"], sp, request_id=job["id"]):
        text = out.outputs[0].text
        yield {"delta": text[len(last):]}
        last = text


runpod.serverless.start({"handler": handler, "return_aggregate_stream": True})
```

3.10 AWS Bedrock: Converse API with Fallback Chain, Batch Inference, and Prompt Caching

```
from __future__ import annotations

import json

import boto3
from botocore.config import Config
from botocore.exceptions import ClientError

br = boto3.client("bedrock-runtime", region_name="us-east-1", config=Config(retries={
"max_attempts": 3, "mode": "adaptive"}, read_timeout=60))

MODEL_CHAIN = ["us.meta.llama3-1-8b-instruct-v1:0", "us.meta.llama3-3-70b-instruct-v1:0&qu
ot;, "us.anthropic.claude-sonnet-4-5-20250929-v1:0"]


def converse(prompt: str, system: str, tier: int = 0, max_tokens: int = 512) -> tuple[str, dict]:
    last_err: Exception | None = None
    for model_id in MODEL_CHAIN[tier:]:
        try:
            r = br.converse(modelId=model_id,
                            system=[{"text": system}, {"cachePoint": {"type"
:: "default"}}] if tier >= 2 else [{"text": system}],
                            messages=[{"role": "user", "content": [{"t
ext": prompt}]}],
                            inferenceConfig={"maxTokens": max_tokens, "temperature":
0.2},
                            guardrailConfig={"guardrailIdentifier": "gr-abc123", &quo
t;guardrailVersion": "3"})
            return r["output"]["message"]["content"][0]["text"],
{"model": model_id, **r["usage"]}
        except ClientError as e:
            code = e.response["Error"]["Code"]
            if code in ("ThrottlingException", "ModelTimeoutException", "Service
UnavailableException"):
                last_err = e; continue                    # fall to next model in chain
```

```
            raise
    raise RuntimeError("all models failed") from last_err


def submit_batch(input_s3: str, output_s3: str) -> str:
    """~50% cheaper than on-demand for deferrable workloads (24h SLA)."""
    ctl = boto3.client("bedrock", region_name="us-east-1")
    r = ctl.create_model_invocation_job(jobName="nightly-summaries", roleArn="arn:aws:iam:
:123456789012:role/BedrockBatch",
                                        modelId=MODEL_CHAIN[1],
                                        inputDataConfig={"s3InputDataConfig": {"s3Uri"
;: input_s3}},
                                        outputDataConfig={"s3OutputDataConfig": {"s3Uri&qu
ot;: output_s3}})
    return r["jobArn"]
```

3.11 Unit Economics Calculator: Dedicated vs Serverless vs API

```
"""Compare monthly cost across deployment options for a
 workload profile; prints break-even utilisation."""
from __future__ import annotations

from dataclasses import dataclass


@dataclass(frozen=True)
class Workload:
    req_per_day: int
    in_tokens: int
    out_tokens: int
    peak_to_avg: float = 3.0
    cache_hit: float = 0.0        # semantic cache fraction
    prefix_cached_frac: float = 0.5


@dataclass(frozen=True)
class Offer:
    name: str
    in_price_per_m: float = 0.0
    out_price_per_m: float = 0.0
    cached_in_price_per_m: float | None = None
    gpu_hr: float = 0.0
    gpus: int = 0
    tps_per_gpu: float = 0.0           # sustained output tok/s at target SLO
    prefill_tps_per_gpu: float = 40_000
    serverless_premium: float = 1.0


def monthly(w: Workload, o: Offer) -> dict[str, float]:
    reqs = w.req_per_day * 30 * (1 - w.cache_hit)
    tin, tout = reqs * w.in_tokens, reqs * w.out_tokens
    if o.gpu_hr == 0:                                  # token-priced API
        cin = o.cached_in_price_per_m if o.cached_in_price_per_m is not None else o.in_price_per_m
        c = (tin * (1 - w.prefix_cached_frac) * o.in_price_per_m + tin * w.prefix_cached_frac * cin + tou
t * o.out_price_per_m) / 1e6
        return {"cost": c, "util": float("nan")}
    peak_tps = (tout / (30 * 86400)) * w.peak_to_avg
    peak_prefill = (tin * (1 - w.prefix_cached_frac) / (30 * 86400)) * w.peak_to_avg
    gpus_needed = max(peak_tps / o.tps_per_gpu, peak_prefill / o.prefill_tps_per_gpu)
    gpus = max(o.gpus, int(gpus_needed) + 1)
    if o.serverless_premium > 1.0:                     # billed per busy-second
        busy_s = tout / o.tps_per_gpu + tin * (1 - w.prefix_cached_frac) / o.prefill_tps_per_gpu
        return {"cost": busy_s / 3600 * o.gpu_hr * o.serverless_premium, "util": 1.0,
 "gpus_peak": gpus}
    cost = gpus * o.gpu_hr * 24 * 30
```

```
    util = (tout / o.tps_per_gpu / gpus) / (30 * 86400)
    return {"cost": cost, "util": util, "gpus": gpus}


if __name__ == "__main__":
    w = Workload(req_per_day=400_000, in_tokens=3500, out_tokens=250, cache_hit=0.25)
    for o in (Offer("API-small", 0.15, 0.60, 0.075),
              Offer("Dedicated H100 8B-FP8", gpu_hr=3.0, tps_per_gpu=5000, prefill_tps_per_gpu=
60_000),
              Offer("Dedicated spot-mix", gpu_hr=1.35, tps_per_gpu=5000, prefill_tps_per_gpu=60
_000),
              Offer("Serverless H100", gpu_hr=3.0, tps_per_gpu=5000, prefill_tps_per_gpu=60_000
, serverless_premium=2.8)):
        print(f"{o.name:<26}", {k: round(v, 2) for k, v in monthly(w, o).items()})
```

3.12 TP vs PP Launch Examples

```
# 70B BF16 on one 8×H100 NVLink node: TP=8 (lowest latency)
vllm serve meta-llama/Llama-3.3-70B-Instruct --tensor-parallel-size 8 --max-model-len 16384 --enable-pref
ix-caching

# 70B FP8 on 2×H100: TP=2 (half the GPUs, ~same quality; cheaper per token)
vllm serve neuralmagic/Llama-3.3-70B-Instruct-FP8-dynamic --tensor-parallel-size 2 --kv-cache-dtype fp8

# 405B BF16 across 2 nodes (8×H100 each): TP=8 within node × PP=2 across nodes (Ray backend)
ray start --head --port=6379                    # node 1
ray start --address=<head-ip>:6379              # node 2
vllm serve meta-llama/Llama-3.1-405B-Instruct-FP8 --tensor-parallel-size 8 --pipeline-parallel-size 2 \
    --distributed-executor-backend ray --max-model-len 8192

# PCIe-only 4×L40S box: avoid TP>2; run 2 replicas of TP=2 (data parallel) behind the gateway
```

---

4. Hard-Learned Production Failures & Edge Cases

Incident 1 — Semantic cache served another customer's answer (cross-tenant leak)

Symptom. A customer received an answer quoting another customer's contract terms.

Root-cause analysis. 1. The cache key was the query embedding only; no tenant/namespace in the index query. 2. "What is my notice period?" is semantically identical across tenants (sim 0.99) → top hit belonged to tenant A. 3. The answer was RAG-derived from tenant A's documents; caching was enabled for all routes.

Remediation. Mandatory namespace tag in every lookup (tenant|model|prompt_v|index_v|context_fingerprint), enforced in the cache API (no un-namespaced calls compile); cacheable=false for personalised/RAG-over-private-docs answers unless namespaced by tenant *and* document-version fingerprint; PII scan before write; periodic cache audit job with canary queries across tenants. Add a test that identical queries from two tenants never share an entry.

Incident 2 — Semantic cache false hits on negation and numbers

Symptom. "Can I get a refund after 30 days?" returned the cached answer for "…within 30 days?" ("Yes"). Complaint volume up; CSAT down.

Root cause. τ = 0.88 set by intuition; small embedding models score such pairs ≥ 0.93. No hard-negative set.

Remediation. Calibrate τ on labelled hard negatives (§3.2 pick_tau) with FPR ≤ 0.5 %; add a validator (cheap intent-equivalence check) for borderline similarity; add a **number/entity guard** (extract digits, dates, named entities from both queries; reject hit if sets differ); negation guard; shorter TTLs for policy-sensitive intents; maintain a "never cache" intent list (money, medical, legal).

Incident 3 — Router collapsed to the small model and quality silently dropped

Symptom. After tuning the router to reduce spend, the cost per request fell 72 % and CSAT dipped 6 points weeks later.

Root cause. The router's training labels came from a judge that preferred longer answers; threshold set on MT-Bench-like data, not production prompts; no online quality monitoring per route.

Remediation. Evaluate routers on **production-distribution** prompts with calibrated judges + human audit; report quality retention per slice (intent, language, difficulty); shadow-evaluate a 2–5 % sample through the large model and compare ("route audit"); online guard: monitor per-route thumbs-down and escalation rate, auto-raise θ when degradation exceeds budget; keep an easy rollback flag.

Incident 4 — Spot reclaim storm took out 70 % of capacity

Symptom. At 09:12 a cloud capacity event reclaimed most spot GPUs in one AZ; p95 TTFT went to 20 s and the queue overflowed.

Root cause. A single instance type in a single AZ; no on-demand floor; autoscaler waited on provisioning for the same unavailable type; weights downloaded from the internet on boot (6 minutes).

Remediation. Diversify across ≥ 3 instance families and AZs (Karpenter/ASG mixed instances policy, capacity-optimised allocation); on-demand baseline sized to critical load; node-local NVMe weights/AMI pre-bake or a regional model cache (cold start < 2 min); gateway **degradation ladder** (shed to smaller model/serverless endpoint, lower max_tokens, queue with 429 Retry-After for batch); chaos drill (terminate 50 % of spot nodes) in staging quarterly.

Incident 5 — Serverless cold starts destroyed p99 and the savings

Symptom. Serverless endpoint had p99 TTFT of 48 s; the bill was *higher* than a dedicated GPU.

Root cause. Traffic of ~1 request/min with scaledown_window=30s: each request cold-started a container (model load 40 s) and billed the GPU for load + 30 s idle — effective utilisation of 3 % billed at a 2.8× premium.

Remediation. Compute break-even (§2.3): at this load use a small always-on dedicated GPU or a hosted per-token API; if serverless, raise scaledown_window to cover inter-arrival gaps, keep min_containers=1 in business hours, use GPU memory snapshots/model volume, route latency-sensitive traffic elsewhere and keep serverless for bursty batch jobs.

Incident 6 — TP=8 on PCIe made things slower and more expensive

Symptom. Moving a 13B model from 1 GPU to TP=4 on an 8×L40S PCIe node *raised* ITL from 22 ms to 31 ms and halved throughput per GPU.

Root cause. All-reduce over PCIe (no NVLink) on every layer; small model gives each GPU tiny GEMMs; synchronisation dominated.

Remediation. Run 8 data-parallel replicas of the (quantised) 13B model on single GPUs behind the gateway; reserve TP for models that don't fit or for NVLink nodes; validate with NCCL all_reduce_perf and a TP sweep benchmark before committing.

Incident 7 — Cascade latency tail: escalations doubled p95

Symptom. Overall mean latency improved 30 % but p95 doubled and users complained about "sometimes very slow" answers.

Root cause. Escalated requests paid T_small + T_verifier + T_large sequentially (~4.5 s vs 2 s).

Remediation. Run the large model **speculatively in parallel** for borderline-difficulty prompts (cancel on small-model success); set a latency budget — if elapsed > X, skip verification; stream the small model's draft only when confidence high; or restrict cascades to non-interactive workloads. Track latency by route and escalation flag.

---

5. Senior/Staff LLMOps Interview Questions

Scenario 1 — "Cut our LLM bill by 60 % without hurting quality. Where do you start?"

Junior/Mid Trap Answer: "Switch to a cheaper model."

Staff Production Answer: "I start with measurement: cost by tenant/feature/model/route and token split (input vs output vs cached), plus quality metrics per slice, so I know where spend concentrates. Then I follow the ladder from least to most risky. (1) Avoid calls: exact and semantic caching with tenant namespacing and calibrated thresholds (20–40 % hit on FAQ-like traffic),

deterministic short-circuits. (2) Shrink tokens: prefix caching by layout (stable first), reranking to cut context from 6k to 3k, rolling-summary memory, output caps by intent. (3) Route: classifier/cascade sending easy traffic to an 8B model, validated by shadow-evaluating samples with a large model, with a quality-retention SLO. (4) Efficiency: FP8/AWQ, chunked prefill, speculative decoding for latency tiers, utilisation raised via better batching. (5) Pricing: spot for batch/burst, committed capacity for baseline, batch APIs for deferrable work. Each step is gated by an A/B or shadow test showing non-inferior quality on CIs, and I'd report $/successful task, not $/token."

Scenario 2 — "Design a semantic cache for a multi-tenant support bot. What can go wrong?"

Junior/Mid Trap Answer: "Embed queries, store answers in Redis, return when similarity > 0.8."

Staff Production Answer: "The functional core is simple; the failure modes are the design. Isolation: every entry is namespaced by tenant, model, prompt version, and index/context version so answers never cross tenants or outlive their sources; entries are tagged with source doc_ids for event-driven invalidation. Accuracy: τ is calibrated on labelled paraphrase vs hard-negative pairs (numbers, negation, entity swaps) to hold false-hit rate ≤ 0.5 %, borderline hits go through a cheap intent-equivalence validator, and number/entity guards reject mismatches. Safety: only cache outputs that passed output rails, no PII, no personalised or tool-derived answers, TTLs by data volatility, and a never-cache intent list. Ops: replay cached text as a stream to preserve UX, monitor hit rate, false-hit complaints, and staleness, and run a periodic cross-tenant canary test. Economics: expected savings $h \cdot c_L - c_lookup - h \cdot f \cdot L_wrong$; I'd check it justifies the complexity at our hit rate."

Scenario 3 — "Tensor vs pipeline parallelism for serving a 405B model?"

Junior/Mid Trap Answer: "Just use as many GPUs as needed with tensor parallelism."

Staff Production Answer: "TP splits every layer and needs an all-reduce twice per layer per token, so it only scales within an NVLink/NVSwitch domain (a single 8-GPU node). 405B in FP8 is ~405 GB — more than one 8×80 GB node's usable memory once KV is included — so I'd either use 8×H200/B200 (141/192 GB each) with TP=8 and FP8, which keeps latency best, or span two nodes with TP=8 within each and PP=2 across, where only activations cross the slower interconnect once per stage. PP adds $(p-1)$ stage latencies to TTFT and bubbles at low load, so it suits throughput-oriented serving with enough concurrency to fill micro-batches. For capacity scale-out I'd add replicas (data parallel) rather than more TP/PP. I'd quantise first (FP8) to reduce the shard count, validate with a TP/PP sweep on real traffic, and size on goodput per dollar. Disaggregated prefill/decode is the next step if TTFT and ITL SLOs conflict."

Scenario 4 — "When is serverless GPU better than a dedicated cluster?"

Junior/Mid Trap Answer: "Serverless is always cheaper because you only pay for what you use."

Staff Production Answer: "Compare effective cost per useful token. Serverless charges a premium per busy-second (k typically 1.5–4×) plus cold-start waste, so break-even utilisation is about 1/k — roughly 25–60 %. Below that (spiky, low average traffic, batch jobs, experiments, long idle periods), serverless wins and removes ops. Above it (steady high load, strict TTFT SLOs, large models with minute-scale cold starts), dedicated or reserved capacity wins, especially with spot for burst. Also weigh: cold-start latency vs SLO, concurrency limits and streaming behaviour, data governance, GPU availability, and engineering overhead. A common architecture is a dedicated baseline sized for the steady floor, serverless or spot for peaks and batch, plus a provider API as the last-resort overflow."

Scenario 5 — "How would you build a router between an 8B and a 70B model?"

Junior/Mid Trap Answer: "Use keywords or prompt length to decide."

Staff Production Answer: "Start from data: for a few thousand production prompts, collect answers from both models and a calibrated judge's preference or a reference-based score; label whether the small model suffices. Train a lightweight router (embedding + logistic regression/gradient boosting, or fine-tuned small encoder à la RouteLLM) that outputs P(small suffices). Choose the threshold on a Pareto curve of quality-retention vs call fraction to the large model, with slice-level checks (language, domain, long context). In production add a cascade safety net — confidence signals (token log-probs, self-consistency, JSON/citation validity) and an optional verifier decide escalation — while controlling the latency tail (parallel speculative large call for borderline cases). Add budget-aware degradation per tenant, shadow evaluation of 2–5 % of routed-small traffic through the large model to detect drift, per-route quality dashboards, and a kill-switch. The router itself must be cheap (<10 ms) and its errors measured."

Scenario 6 — "GPU utilisation is 30 %; the CFO wants the fleet cut in half. What do you do?"

Junior/Mid Trap Answer: "Cut the fleet in half since it's mostly idle."

Staff Production Answer: "'GPU util' isn't capacity headroom: it can't see KV-cache pressure, queue tail, or diurnal peaks. First I'd characterise load: peak-to-average, p99 concurrency, KV usage, queue depth, TTFT/ITL vs SLO by hour. If peaks need the capacity, the low average comes from diurnal valleys — fix with autoscaling on queue/KV signals, scheduled scaling, consolidating low-traffic models, and filling valleys with batch/offline work (evals, synthetic data, embeddings) on the same GPUs or on spot. Right-size: quantise (FP8/AWQ) to fit smaller or fewer GPUs, tune max-num-seqs/batch settings (under-batching is common), enable prefix caching, and move small models to cheaper GPUs (L4/A10). Reduce demand: semantic caching and routing. Then cut only what simulation shows still meets SLOs with N+1 redundancy, and commit baseline capacity via reservations."

Scenario 7 — "Estimate the cost per 1M output tokens for Llama-3 70B on 4×H100."

Junior/Mid Trap Answer: "Take the GPU price per hour and divide by tokens."

Staff Production Answer: "I'd compute it from measured throughput at the SLO, not peak benchmark numbers. Fleet cost: 4 × $3/hr = $12/hr. Suppose FP8 TP=4 sustains ~2,800 output tok/s at the target ITL with realistic prompts. At 100 % busy: 12 / (2800×3600) × 1e6 ≈ $1.19 per 1M output tokens. At 40 % billing utilisation the effective price is ~$2.98 — idle time is the real cost. I'd add prefill (input tokens cost GPU time too; prefill ~10–20× cheaper per token than decode but dominates for long RAG prompts), so I'd compute a blended cost using separate prefill and decode throughput, and account for overheads (redundancy, headroom, networking, engineering). Then I'd compare to API pricing at our actual volume and utilisation to decide self-host vs buy, and note the sensitivity: FP8, speculative decoding, and better batching shift the number by 1.5–2×."

Scenario 8 — "Our agent costs $0.40 per task; product wants $0.05. How do you get there?"

Junior/Mid Trap Answer: "Use a cheaper model for the whole agent."

Staff Production Answer: "Break the $0.40 down with traces: tokens by step, number of LLM calls, retries, tool output sizes, and context growth (quadratic without caching). Typical wins: (1) cap and summarise tool outputs, keep history append-only for prefix caching (cuts input cost ~70 %); (2) replace LLM micro-decisions with deterministic code or merged tool calls to reduce step count; (3) route sub-steps — planning/final answer on the large model, extraction/classification/tool-arg filling on an 8B or fine-tuned small model; (4) distil the successful trajectories into a small fine-tuned model; (5) semantic caching for repeated sub-queries and tool results; (6) early termination and loop breakers; (7) batch non-urgent subtasks. I'd track cost per *successful* task and keep a quality gate (task success rate non-inferior) while iterating."

Key Takeaways

- Cost ladder: **avoid calls** → **fewer tokens** → **cheaper model** → **more tokens per GPU-second** → **cheaper GPU-hours.**
- **Utilisation** is the master variable; compute break-evens (dedicated vs serverless vs API) with real numbers.
- Semantic caches need **namespacing, calibrated τ, validators, and safe write-back**; routers need **production-distribution evaluation and shadow audits**.
- TP within NVLink, PP across nodes, replicas to scale; quantise before sharding.
- Spot needs diversification, drain logic, fast cold starts and a degradation ladder.

11.6 GPU Capacity and Procurement: Commitments, Sharing (MIG/MPS) and Bin-Packing

Conceptual Deep Dive

GPU spend is dominated by how you buy and how you pack, not only by kernel efficiency. Purchase models: on-demand (flexible, highest unit price), reserved or committed use (discount for a term), spot or preemptible (large discount, can be reclaimed), capacity blocks (reserved windows for short intensive jobs) and specialist GPU clouds (different pricing, fewer managed services). The central risk of any commitment is that model sizes, quantisation choices and serving engines change faster than contract terms; a commitment sized to today's 70B deployment can be wrong after a distilled 8B model ships (15.5).

Sizing starts from the demand curve: diurnal and weekly shapes, p99 peaks versus average, plus surge for rollouts (old and new versions run together) and failover. A typical structure is a committed baseline near a lower-to-middle percentile of demand, on-demand for the predictable shoulder, and spot or preemptible for tolerant batch work. The optimum commits where the probability that demand exceeds capacity equals one minus the discount ratio (a newsvendor argument), so it is rarely 100% of peak.

Match GPU type to bottleneck. Decode is memory-bandwidth bound; long contexts and many concurrent sequences are memory-capacity bound through the KV cache; prefill and large batches are compute bound. Pick memory capacity and bandwidth for chat serving, compute for batch scoring.

Sharing. MIG (on supported data-centre GPUs) partitions a GPU into hardware-isolated slices with fixed memory and compute; MPS and time-slicing share a device with no memory isolation, so one tenant can starve or crash another. Many small or rarely used models on whole GPUs waste money; consider MIG, multi-adapter serving (Module 05 section 1.7), or scale-to-zero with warm pools. Check current vendor docs for supported partitions.

Bin-packing. A replica needs weights + KV pool + activation and runtime overhead; pack replicas by memory with headroom for KV growth and fragmentation, using first-fit-decreasing as a baseline. **Metrics:** goodput (requests meeting SLO per second) and tokens per second per dollar; SM utilisation alone misleads because a busy GPU can be memory-stalled or running tiny batches. Procurement lead times for large reservations can be weeks to months (verify with the provider), so forecasting is part of the job.

Edge cases: a commitment sized for today's model while next quarter's needs twice the memory; burst capacity unavailable in your region exactly when you need it, so test quota and keep a second region or provider path.

Cross-Connection Scenarios

- KV-cache sizing and batch limits (Module 05, Module 01 section 2.5).
- Autoscaling signals and scale-to-zero (Module 11 legacy autoscaling).
- Distilled or quantised models changing the footprint (15.5, Module 04).
- Unit economics and idle-cost allocation (11.8).

Production Anti-Patterns

- Committing to 100% of peak: you pay the discounted rate for hours of idle capacity every night.
- Packing replicas with no headroom for KV growth: out-of-memory evictions under longer prompts.
- Reading nvidia-smi utilisation as efficiency: it reports time with any kernel active, not useful throughput.

Minimal Working Code

A first-fit-decreasing bin-packer placing model replicas by memory onto GPUs, reporting GPUs used, fragmentation and cost (the hourly rate is a placeholder).

```
GPU_GB = 80
HEADROOM = 0.10             # fraction kept free for KV growth
RATE = 2.0                  # placeholder $/GPU-hour, perishable

def pack(replicas):
    cap = GPU_GB * (1 - HEADROOM)
    gpus = []               # each: {"free": float, "items": [...]}
    for name, gb in sorted(replicas, key=lambda r: -r[1]):
        for g in gpus:
            if g["free"] >= gb:
                g["free"] -= gb
                g["items"].append(name)
                break
        else:
            if gb > cap:
                raise ValueError(f"{name} needs {gb} GB, exceeds usable {cap}")
            gpus.append({"free": cap - gb, "items": [name]})
    used = sum(cap - g["free"] for g in gpus)
    frag = 1 - used / (len(gpus) * GPU_GB)
    return gpus, frag, len(gpus) * RATE

replicas = [("llm-a", 40), ("llm-b", 40), ("small-1", 20), ("small-2&q
uot;, 20),
            ("small-3", 15), ("small-4", 10), ("small-5", 10)]
gpus, frag, cost = pack(replicas)
print(len(gpus), "GPUs", f"fragmentation={frag:.2f}", f"cost/h={cost}")
assert len(gpus) == 3 and all(g["free"] >= 0 for g in gpus)
assert sum(len(g["items"]) for g in gpus) == len(replicas)
```

A commitment optimiser sweeping committed GPUs against an hourly demand curve with committed, on-demand and spot placeholder rates.

```
import numpy as np

rng = np.random.default_rng(1)
hours = np.arange(24 * 7)
demand = 40 + 25 * np.maximum(0, np.sin((hours % 24 - 6) / 24 * 2 * np.pi)) + rng.normal(0, 3, hours.size
)
demand = np.ceil(np.maximum(demand, 0))

COMMITTED, ON_DEMAND, SPOT = 0.6, 1.0, 0.35      # placeholder $/GPU-hour, perishable
SPOT_SHARE = 0.3                                 # share of burst placed on spot

def weekly_cost(c):
    burst = np.maximum(demand - c, 0).sum()
    return c * COMMITTED * demand.size + burst * (SPOT_SHARE * SPOT + (1 - SPOT_SHARE) * ON_DEMAND)

grid = np.arange(0, int(demand.max()) + 1)
costs = np.array([weekly_cost(c) for c in grid])
best = int(grid[costs.argmin()])
print("best commitment", best, "peak", int(demand.max()), "mean", round(dem
and.mean(), 1))
assert best < demand.max()
assert costs.min() < weekly_cost(int(demand.max()))
assert costs.min() < weekly_cost(0)
```

Interview Angle

Q: How much GPU capacity should you commit to versus rent on demand? **A:** Fit a baseline to the demand percentile where marginal committed cost equals expected on-demand cost, typically well below peak, then cover the shoulder with on-demand and tolerant batch with spot. Keep terms short relative to model churn. Trade-off: discount versus flexibility. Check: simulate the last 90 days of hourly demand against the plan and report weekly cost versus all-on-demand and the hours of unmet demand.

Q: You have 40 small fine-tuned models, each used rarely. How do you host them? **A:** Prefer one base model with multi-adapter serving (LoRA hot-swapping) over 40 GPUs, with adapters loaded on demand and a warm set for hot tenants. If full fine-tunes are needed, use MIG slices or scale-to-zero with cold-start tolerance. Trade-off: cold-start latency and noisy neighbours. Check: p95 time-to-first-token for cold versus warm adapters and cost per adapter per month.

Q: GPU utilisation is 85% but cost per token is rising. Explain. **A:** Utilisation counts busy time, not efficient work. Causes: smaller batches from lower concurrency, longer prompts or larger KV reducing batch size, memory-bound decode stalls, retries and wasted speculative work, a less efficient engine version, or cost allocation changes. Check goodput, batch size distribution, tokens per second per GPU and HBM bandwidth, and compare before and after the cost rise.

11.7 Provider-Side Economics: Rate Limits, Adaptive Concurrency, Batch APIs and Multi-Provider Failover

Conceptual Deep Dive

Hosted APIs add cost and reliability levers that self-hosting does not. Cost levers: **prompt caching** (keep stable prefixes at the start of the prompt; check each provider's current rules and discounts), a **batch API** for deferrable work (typically a discount in exchange for asynchronous completion within a window; verify current terms), model tiers and routing (11.8, Module 11 legacy routing), output-length caps, and request deduplication.

Rate limits are usually expressed as requests and tokens per minute within usage tiers. A 429 means slow down: honour a Retry-After header when present, otherwise exponential backoff with full jitter. A fixed thread pool is the wrong tool because the right concurrency changes with provider load. **Adaptive concurrency** (AIMD, as in TCP congestion control) increases the in-flight limit additively on success and cuts it multiplicatively on 429 or timeout, converging near the true capacity. Separate limits per model and per key, and budget tokens, not just requests.

Multi-provider failover. A second provider is not a drop-in: prompts and tool schemas may behave differently, tokenisers change token counts and cost, safety filters refuse different things, JSON-mode guarantees differ, and each provider has its own

data-processing terms (a fallback that violates residency or contract is a compliance incident). Evaluate the fallback on your eval set (legacy 1.8) and keep it exercised with a small continuous traffic share or synthetic probes. Use **circuit breakers** (closed, open, half-open) with health scores, and optionally **hedged requests** (send a duplicate after a latency threshold) which trade cost for tail latency.

Budgets per key and tenant with spend alerts, and usage forecasting from token telemetry. **Provisioned throughput** (reserved capacity) is worth it when sustained utilisation exceeds the break-even fraction: the committed price divided by the pay-as-you-go price for the same tokens. Below that utilisation you pay for idle capacity.

Edge cases. When an outage ends, every client retries at once (thundering herd): add jitter, ramp concurrency gradually and use half-open probes. Retries on streaming responses that fail midway can pay for the partial output twice; resume or accept partial results where possible, and never auto-retry non-idempotent tool calls.

Cross-Connection Scenarios

- Gateway design and routing (Module 11 legacy gateway, 1.8).
- Idempotency of tool calls under retries (9.8).
- Compliance of fallback providers and data residency (8.7).
- Spend alerts and unit economics (11.8).

Production Anti-Patterns

- Fixed-size thread pool hammering a 429ing endpoint: wasted calls, longer queues and possible key throttling.
- Fallback provider never exercised until an incident: the first real use discovers broken schemas and unapproved terms.
- Retrying non-idempotent tool calls at the gateway: duplicate side effects such as double refunds.

Minimal Working Code

An AIMD adaptive concurrency limiter simulated against a rate-limited fake provider and compared with fixed concurrency.

```
CAPACITY = 20          # requests the provider accepts per tick

def simulate(controller, ticks=300):
    ok = limited = 0
    for _ in range(ticks):
        n = controller.limit()
        accepted = min(n, CAPACITY)
        rejected = n - accepted
        ok += accepted
        limited += rejected
        controller.feedback(rejected > 0)
    return ok, limited

class Fixed:
    def __init__(self, n): self.n = n
    def limit(self): return self.n
    def feedback(self, throttled): pass

class AIMD:
    def __init__(self, add=1, mult=0.5):
        self.c, self.add, self.mult = 1.0, add, mult
    def limit(self): return int(self.c)
    def feedback(self, throttled):
        self.c = max(1.0, self.c * self.mult) if throttled else self.c + self.add

for name, ctl in [("fixed-50", Fixed(50)), ("fixed-10", Fixed(10)), ("aimd"
, AIMD())]:
    ok, lim = simulate(ctl)
    print(name, "success", ok, "429s", lim)
ok_a, lim_a = simulate(AIMD())
ok_f, lim_f = simulate(Fixed(50))
ok_s, _ = simulate(Fixed(10))
assert lim_a < lim_f and ok_a > ok_s
```

A failover router with health scores, circuit-breaker states and cost-aware ordering, tested deterministically with fake providers.

```
class Provider:
    def __init__(self, name, cost, healthy=True):
        self.name, self.cost, self.healthy, self.calls = name, cost, healthy, 0
        self.score, self.fails, self.opened_at = 1.0, 0, None
    def call(self, prompt):
        self.calls += 1
        if not self.healthy:
            raise RuntimeError("provider down")
        return f"{self.name}:ok"

class Router:
    def __init__(self, providers, threshold=3, cooldown=10):
        self.ps, self.threshold, self.cooldown = providers, threshold, cooldown
    def _allowed(self, p, now):
        return p.opened_at is None or now - p.opened_at >= self.cooldown   # open -> half-open afte
r cooldown
    def call(self, prompt, now):
        for p in sorted(self.ps, key=lambda p: (p.score < 0.5, p.cost)):
            if not self._allowed(p, now):
                continue
            try:
                out = p.call(prompt)
                p.fails, p.opened_at, p.score = 0, None, 0.8 * p.score + 0.2
                return out
            except RuntimeError:
                p.fails += 1
                p.score *= 0.8
                if p.fails >= self.threshold:
                    p.opened_at = now
        raise RuntimeError("all providers failed")

cheap, premium = Provider("cheap", 1, healthy=False), Provider("premium", 5)
r = Router([cheap, premium])
for t in range(3):
    assert r.call("hi", now=t) == "premium:ok"
assert cheap.opened_at is not None and cheap.calls == 3
r.call("hi", now=4)
assert cheap.calls == 3                    # breaker open: not called
cheap.healthy = True
assert r.call("hi", now=20) == "cheap:ok"    # half-open probe succeeds, cheap prefer
red again
print("router checks passed")
```

Interview Angle

Q: Your batch job keeps hitting 429. Redesign it. **A:** Move to the provider's batch API if latency allows; otherwise replace the fixed pool with AIMD concurrency plus token-aware rate limiting, exponential backoff with jitter honouring Retry-After, a work queue with checkpointing, and idempotent job ids. Trade-off: throughput variance and complexity. Check: 429 rate under 1% of calls, steady throughput near the quota, and no duplicate outputs after restarts.

Q: How do you design failover between two LLM providers safely? **A:** Evaluate the secondary on the same eval set, adapt prompts and tool schemas per provider behind one interface, confirm data terms and region, use circuit breakers with health scores, and exercise it continuously with a small traffic share. Trade-off: extra maintenance and slight quality drift. Check: eval score gap within an agreed margin, failover time under target in a game day, and cost impact.

Q: When is provisioned throughput worth it? **A:** When sustained utilisation of the reserved capacity exceeds the break-even fraction (committed price divided by on-demand price for equal tokens) and latency or throttling guarantees matter. Otherwise pay-as-you-go plus batch. Verify current terms. Check: 30-day token telemetry by hour, utilisation percentile versus break-even, and tail latency under peak.

11.8 LLM FinOps: Unit Economics, Showback, Forecasting and Efficiency Metrics

Conceptual Deep Dive

FinOps for LLM systems turns an invoice into decisions. The invoice total is useless for steering; **unit metrics** are what you manage: cost per successful task, per resolved ticket, per active user, per thousand requests, and gross margin per feature. Cost per request can fall while cost per successful task rises if quality dropped and users retry.

Tagging is the prerequisite: every call carries tenant, feature, route, model, environment and bundle id (a log record or gateway header, not just cloud billing tags). **Showback** reports costs to teams; chargeback bills them. Shared costs, notably idle GPU time, are allocated by a stated rule such as usage share (GPU-seconds), so idle capacity is not charged to the loudest team and the total always reconciles to the invoice.

Forecasting. Drivers are token growth per user, seasonality, new feature launches and price changes. Prices are perishable, so model them as scenarios (flat, down 30%, up 10%) rather than a point estimate. Set budgets with alerts at forecast bands. **Anomaly detection** on daily cost per tenant and feature should use robust statistics: median and MAD with a day-of-week baseline resist the very spikes you want to catch, unlike mean and standard deviation. An agent-loop cost incident (Module 10 legacy incident) is exactly this shape.

Efficiency KPIs: tokens per task, cache hit rate, share of traffic served by the small model, batch-API share, GPU goodput per dollar. Keep an optimisation backlog ranked by expected savings times (1 - risk), and gate every optimisation on a quality guardrail (Module 12 incident 3). An optional sustainability metric is estimated energy per million tokens; grid carbon factors vary by region and time, so report assumptions.

Edge cases. Shared prefix caches and multi-tenant batches make per-tenant GPU attribution approximate; document the allocation method and its error. Free-tier abuse looks like organic growth in token usage, so compare against signup and engagement signals and add per-account quotas.

KPI	Decision it drives
Cost per successful task	Model and prompt choices
Cache hit rate	Prompt structure, TTLs
Small-model traffic share	Routing thresholds
Idle GPU share	Autoscaling and commitment

Cross-Connection Scenarios

- Tail sampling and cost metrics from telemetry (10.6).
- Capacity and commitment choices feeding allocation (11.6).
- Provider batch discounts and caching (11.7).
- Quality guardrails for cost optimisation (Module 12, 14.4).

Production Anti-Patterns

- Reporting only monthly invoice totals: spikes surface weeks late and nobody can attribute them.
- Allocating idle cost to the loudest team: incentives distort and the rule is contested.
- Optimising cost with no quality guardrail: a cheaper model quietly lowers success rate and revenue.

Minimal Working Code

A showback allocator splitting GPU-hour cost across tenants by GPU-seconds, with idle cost allocated proportionally and a reconciliation check (the rate is a placeholder).

```
def showback(total_cost, capacity_seconds, usage_seconds):
    busy = sum(usage_seconds.values())
    idle = capacity_seconds - busy
    idle_cost = total_cost * idle / capacity_seconds
    rows = {}
    for tenant, sec in usage_seconds.items():
```

```
        direct = total_cost * sec / capacity_seconds
        shared_idle = idle_cost * sec / busy
        rows[tenant] = {"direct": direct, "idle_share": shared_idle, "total"
;: direct + shared_idle}
    return rows

RATE_PER_GPU_HOUR = 2.0                    # placeholder, perishable
gpus, hours = 4, 24
total = gpus * hours * RATE_PER_GPU_HOUR
capacity = gpus * hours * 3600
usage = {"tenant-a": 150_000, "tenant-b": 90_000, "tenant-c": 30_000}
rows = showback(total, capacity, usage)
for t, r in rows.items():
    print(f"{t:9s} direct={r['direct']:.2f} idle={r['idle_share']:.2f} total={r['total']:.2f}")
assert abs(sum(r["total"] for r in rows.values()) - total) < 1e-6
assert rows["tenant-a"]["total"] > rows["tenant-c"]["total"]
```

A robust cost-anomaly detector (median and MAD with a day-of-week baseline) over daily per-tenant cost.

```
import numpy as np

rng = np.random.default_rng(3)
days = 56
weekday_level = np.array([100, 110, 112, 108, 105, 60, 55], dtype=float)
cost = np.array([weekday_level[d % 7] for d in range(days)]) + rng.normal(0, 4, days)
cost[40] += 120                            # injected agent-loop style spike

def anomalies(cost, z=3.5):
    dow = np.arange(len(cost)) % 7
    baseline = np.array([np.median(cost[dow == d]) for d in range(7)])
    resid = cost - baseline[dow]
    mad = np.median(np.abs(resid - np.median(resid))) or 1e-9
    robust_z = 0.6745 * (resid - np.median(resid)) / mad
    return [int(i) for i in np.where(np.abs(robust_z) > z)[0]]

flags = anomalies(cost)
print("flagged days", flags)
assert 40 in flags and len(flags) <= 3
```

Interview Angle

Q: How do you define unit economics for an AI feature and what goes on the executive dashboard? **A:** Cost per successful task (all-in: tokens, retrieval, GPU share, evaluation overhead) against revenue or value per task, giving margin per feature, plus quality and adoption beside it. Executive view: unit margin trend, spend versus forecast band, top cost drivers and savings from the optimisation backlog. Trade-off: attribution approximations. Check: allocated costs reconcile to the invoice within a stated tolerance.

Q: Cost per request fell 30% but margin did not improve. Why? **A:** Candidates: more requests per task (retries, longer agent loops), lower success rate so cost per successful task rose, shifting traffic to lower-priced but lower-value tiers, idle capacity unchanged, or revenue per task falling. Check cost per successful task, requests per task, success rate by route and idle share before and after the change.

Q: How would you forecast next year's LLM spend given uncertain prices? **A:** Forecast tokens bottom-up (users, tasks per user, tokens per task with seasonality and launches), then apply price scenarios rather than a single price, with efficiency gains modelled separately. Report a range and the drivers, and re-forecast monthly. Trade-off: range width versus decision usefulness. Check: back-test the method on past quarters and track forecast error (MAPE).